

Season and Temperature Analysis Program Report

ISAD1000- Assignment

Name: Isini Ayansa Perera

Student Id: 23601321

1. Introduction

This report explains the overall design, implementation, testing, and documentation process completed for this assignment. The project was mainly developed using Java and focuses on two systems called SeasonFinder and TemperatureChecker. Different modules and submodules were created by applying the modularity concepts taught during lectures and worksheets.

Throughout this assignment, production code was implemented together with different testing methods such as equivalence partitioning, boundary value analysis, and white-box testing. Java test harnesses were also created to execute and verify the test cases. Git version control was used during the development process to manage and track changes made to the project files.

This report includes the module descriptions, implementation details, testing designs, test execution results, version control activities, and a final discussion about the work completed, challenges faced, and possible future improvements.

2. Production code design-Module Description

The modules were designed by dividing the program into submodules where each module performs a specific task. This helped to improve readability, testing, debugging, and maintenance of the program. Validation modules such as isValidCountry() and isValidTemp() were separated to reduce repeated code and improve reusability.

Some assumptions were also made during the implementation. Only the countries and cities stored in the program are considered valid inputs. Months are expected to be between 1 and 12, while temperatures must remain within the valid ranges defined for each city.

2.1 summary

Module	Purpose
Menu	Handles user interactions and navigates the menu
SeasonFinder	Finds season, compares seasons
TemperatureChecker	temperature calculations and file handling

2.2 SeasonFinder Module

Module1: isValidCountry()

submodule	isValidCountry()
Purpose	Checks whether the given country name is valid by the system
Imports	country(String)
Exports	valid(boolean)
Behaviour	Receives the country as a parameter and if its uppercase, it converts to lowercase and checks whether it is valid or now, and it preturns false if its invalid
Dependancies	(None)
Exceptions	(None)

Module2: isValidMonth()

submodule	isValidMonth()
Purpose	checks whether the given month is valid by the system
imports	month(integer)
exports	valid(boolean)
behaviour	Receives the month as a parameter and checks whether the its within the valid range or not , and it returns false, if its invalid.
dependancies	(None)
Exceptions	(None)

Module3: findMeteorologicalSeason()

submodule	findMeteorologicalSeason()
Purpose	find meteorological season by comparing month and country entered with the valid months and countries by the system
imports	country(String), month(Integer)
exports	meteorologicalSeason(String)
behaviour	When user enters valid country and month, country is converted to lowercase and compare the month with the defined seasons of each country, and returns the correct meteorological season to the user
dependancies	isValidCountry(), isValidMonth()
exceptions	IllegalArgumentException

Module4: findTraditionalSeason()

submodule	findTraditionalSeason()
purpose	find noongar season by comparing month and country entered with the valid months and countries by the system
imports	country(String), month(Integer)
exports	traditionalSeason(String)
Behaviour	When user enters valid country and month, country is converted to lowercase and compare the month with the defined seasons of australia, and returns the correct traditional season to the user
dependancies	isValidCountry(), isValidMonth()
exceptions	IllegalArgumentException

Module5: countrySeasonCompare()

submodule	countrySeasonCompare()
purpose	compares the meteorological seasons of the user entered two countries by the system
imports	country1(String), country2(String), month(Integer)
exports	comparisonResult(String)
Behaviour	When the user enters 2 valid countries and a month, calls the findMeteorologicalSeason()for both entered countries, compare the seasons and returns a suitable message if seasons are same or not to the user
dependancies	isValidCountry(), isValidMonth(), findMeteorologicalSeason()
exceptions	IllegalArgumentException

2.3 TemperatureChecker Module

Module1: isValidCity()

submodule	isValidCity()
purpose	checks whether the given city is valid by the system
imports	city(String)
exports	valid(boolean)
behaviour	Gets the city from the user and if its uppercase, it converts to lowercase and checks whether it is valid or not by comparing against the stored cities, and it returns false if its invalid
dependancies	(None)
exceptions	(None)

Module2: isValidTemp()

submodule	isValidTemp()
purpose	Checks whether the entered temperature is valid
imports	city(String), temp(double)
exports	valid
behaviour	Gets the temperature and city from the user, validates the city and compares the temperature entered against the min and max temperature ranges predefined for each city
dependancies	isValidCity()
exceptions	(None)

Module3: findAvgTemp()

submodule	findAvgTemp()
purpose	average temp is found based on the entered city and time of the day
imports	city(String), time(String)
exports	averageTemp(double)
behaviour	Gets the city and time from the user, converts them to lowercase and compares them with the predefined average temeperature values and return the correct average temperature
dependancies	(None)
exceptions	(None)

Module4: compareAvgTemp()

submodule	compareAvgTemp()
purpose	compares the user entered temperature with the average temp of the city the user enetered
imports	city(String), temp(double), time(String)
exports	comparisonResult(String)
behaviour	When the user enters city, temperature and time, it validats and calculate the difference of the entered temperature and the average temperature of the entered city. The system will return a suitable message stating that whether the temperature is above, below or equal to average temperatuer. If the difference is greater than 6 degrees celsius, a message will be printed by the system
dependancies	isValidCity(), isValidTemp(),findAvgTemp()

submodule	compareAvgTemp()
exceptions	IllegalArgumentException

Module5: comparePerthTemp()

submodule	comparePerthTemp()
purpose	compares the user entered temperature with the Perth average temp and stores the results in a "perthComparison.txt"
imports	city(String), temp(double)
exports	perthComparison.txt
behaviour	Verifies the city and temperature values entered by the user, calculates the difference between the entered temperature and both the morning and afternoon temperatures in Perth and creates appropriate comparison messages, and uses file handling to record them into "perthComparison.txt."
Dependencies	isValidCity(), isValidTemp(), FileWriter
exceptions	IllegalArgumentException, IOException

Module6: readPerthFile()

submodule	readPerthFile()
purpose	To display the contents of the 'perthComparison.txt' file
imports	perthComparison.txt
exports	displays the contents of the file onto the screen
behaviour	The perthComparison.txt file is read line by line using BufferedReader and displays each line on the screen until there are no line to read. An error message would be printed if the file is not able to be found by the sustem
dependencies	BufferedReader, FileReader
exceptions	FileNotFoundException, IOException

2.4 Menu Module

submodule	main()
purpose	prints the menu to the user and handles all the user interactions
imports	choice(Integer), country(String), city(String), month(Integer), temperature(double), time(String)
exports	menu interactions, error messages and method calls
behaviour	displays the menu to the screen until user exits by choosing '0'. Uses scanner to enter user inputs, validates the empty input, invalid numbers, and when choice is selected, based on it, it calls SeasonFinder and TemperatureChecker modules
dependencies	readInt(), SeasonFinder.java, TemperatureChecker.java
exceptions	IllegalArgumentException, NumberFormatException

3. Production code implementation

Running the production code 1. to compile the files by

"javac SeasonFinder.java TemperatureChecker.java Menu.java" 2. to run the files by

"java Menu"

```

isini@isini:~/Perera_Isini_23601321_ISE_Repo2/code$ javac *.java
isini@isini:~/Perera_Isini_23601321_ISE_Repo2/code$ java Menu

=====Seasons and Temperature System=====
1> Find season- Meteorological seasons
2> Find season- Traditional seasons
3> Compare seasons
4> Compare city average temperature
5> Compare temperature of city with Perth
6> View perth comparison txt file
0> Exit
Enter choice:

```

4. Production code Implementation- Review Checklist

Checklist Item	Yes or No
Modules have meaningful names	Yes
Each module performs one specific task	Yes
Inputs and outputs are clearly defined	Yes
Repeated code has been minimized	Yes
Validation is handled properly	Yes
Modules are easy to test and maintain	Yes

4.1 Refactoring Decisions

During the review process, the following issues were identified and fixed:

- Method names '*isCity()*' and '*isTemp()*' were renamed to '*isValidCity()*' and '*isValidTemp()*' to better describe their purpose and improve readability.
- The '*countrySeasonCompare()*' method had a missing return statement in the same season branch which was fixed.
- The '*comparePerthTemp()*' method was missing '*writer.close()*' which caused the file not to save properly. This was fixed.
- Output messages in '*compareAvgTemp()*' were missing spaces before "degrees C"
- A new method '*isValidMonthInput(String)*' was added to handle null and empty String inputs for month before converting to integer.

5. Black-box Test Cases

The following black-box test cases were designed using Equivalence Partitioning(EP) and Boundary Value Analysis(BVA) techniques based on the module descriptions created earlier. The test cases were designed to test both valid and invalid inputs for different modules in the system.

Some assumptions made during testing include:

- Only predefined countries and cities are considered valid inputs.
- Months are expected to be within the range 1 to 12.
- Temperature values should remain within the valid ranges defined for each city.

5.1 Boundary Value Analysis(BVA)

isValidMonth()

category	test data	expected result
month>=1 && month<=12	5	true
month lower boundary	1	true
month upper boundary	12	true
month==0	0	false
month<0	-10	false
month>12	15	false

isValidTemp()

category	test data	expected result
temp>=0.7 && temp<=46.0	perth, 25.0	true
temp lower boundary	perth, 0.7	true
temp upper boundary	perth, 46.0	true
temp<0.7	perth, 0.1	false
temp>46.0	perth, 90	false

5.2 Equivalence Partitioning(SeasonFinder.java)

isValidCountry()

category	test data	expected result
checks country is equal to already stored countries	sri lanka	true
country valid but entered in capitals	SRI LANKA	true
istudent last name as country	Perera	false
checks if country is not equal to already stored countries	italy	false
checks if country is empty	""	false

isValidMonth()

category	test data	expected result
month<13	10	true
month >0	5	true
month>12	15	false
month==0	0	false
month < 0	-10	false
last 3 digits of studentID as month	321	false

findMeteorologicalSeason()

category	test data	expected result
valid northeast monsoon month for sri lanka	sri lanka, 12	Northeast Monsoon
valid winter month for japan	japan, 1	Winter

category	test data	expected result
valid malaysia monsoon	malaysia, 4	Inter Monsoon
valid spain autumn month	spain, 9	Autumn
valid australia spring month	australia, 10	Spring
valid mauritius summer month	mauritius, 3	Summer
invalid country	italy	"Country is unavailable"
valid country and invalid month	australia, 15	"Invalid month... Enter between 1 and 12"

findTraditionalSeason()

category	test data	expected result
valid noongar season month	australia, 1	Birak
valid noongar season for another month	australia, 5	Djeran
Invalid country	france	"Country is unavailable"
Invalid month	australia , 25	"Invalid month.. Enter between 1 and 12..."

countrySeasonCompare()

category	test data	expected result
country1 and country2 having the same season	malaysia, sri lanka, 10	"malaysia and sri lanka have the SAME season"
country1 and country2 having different seasons	sri lanka, japan, 10	"sri lanka and japan don't have the SAME season"
invalid country1	italy	"Country unavailable"
invalid country2	japan, paris	"Country unavailable"
valid countries but invalid month	malaysia, sri lanka, 25	"Invalid month.. Enter between 1 and 12..."

5.3 Equivalence Partitioning(TemperatureChecker.java)

isValidCity()

category	test data	expected result
Valid city	brisbane	true
another valid city	perth	true
invalid city	colombo	false
city empty	""	false

isValidTemp()

category	test data	expected result
valid brisbane temperature	brisbane, 35.0	true
valid perth temperature	perth, 30.0	true
valid hobart temperature	hobart, 20	true
invalid city	colombo, 25.0	false
city temperature above valid range	perth, 78.8	false

category	test data	expected result
city temperature below valid range	hobart, -10.2	false

compareAvgTemp()

category	test data	expected result
temperature is above average and greater than 6 degrees	brisbane, 30, morning	"Temperature is above average by 8.2 degrees c and difference is greater than 6 degrees c"
temperature is above average	perth, 25, afternoon	"Temperature is above average by 2.0 degrees c"
temperature is below average and difference is greater than 6 degrees	perth, 10, afternoon	"Temperature is below average by 13.0 degrees and difference is greater than 6 degrees c "
temperature is below average	hobart, 10, afternoon	"Temperature is below average by 5.1 degrees C"
temperature is equal to average	perth, 23.0, afternoon	"Temperature and Average is equal"
invalid city	colombo	"City is unavailable "
invalid time	brisbane, midnight	"Not a valid time of day"
invalid temperature	perth, 60, morning	"Not a valid temperature range"

comparePerthTemp()

category	test data	expected result
valid temperature above compared to perth morning average	perth, 25	"Temperature is above morning average of Perth by 6.8 degrees C..."
valid temperature below compared to perth morning average	hobart, 10	"Temperature is below the morning average of Perth by 8.2 degrees C..."
temperature being equal to perth afternoon average	perth, 23.0	"Temperature is equal to the afternoon average of Perth..."
invalid city	colombo	"City is unavailable"
invalid temperature	perth, 70	"Not a valid temperature"

6. White-box Test Cases

The following white-box test cases were designed based on the logic and execution paths of selected modules in the program. The test cases mainly focus on if-else paths and nested conditional paths to verify different execution flows within the modules. Assumptions made during testing include:

- Valid cities and countries are predefined inside the program.
- Temperatures are expected to remain within the valid ranges defined for each city.
- The compareAvgTemp() module returns output messages based on the calculated temperature difference.

*** The two different types of constructs tested below are *if-else paths* and *nested if-else paths* in the 'compareAvgTemp()' submodule.**

Path	Test Data	Expected Result
1. Enter diff>0 path	city= perth, temp=25, time=afternoon	output: "Temperature is above average by 2.0 degrees C"
2. Enter diff>6 path	city= brisbane, temp= 30, time= morning	output: "Temperature is above average by 8.2 degrees C and difference is greater than 6 degrees C"
3. Enter diff <0 path	city= hobart, temp=10, time= afternoon	output: "Temperature is below average by 5.1 degrees C"
4. Enter equal path	city= perth, temp= 23.0, time=afternoon	output: "Temperature and Average is equal"
5. Enter invalid city path	city= colombo, temp=20, time=morning	output: "City is unavailable"

the construct tested below is *try-catch path* for 'readPerthFile()' submodule

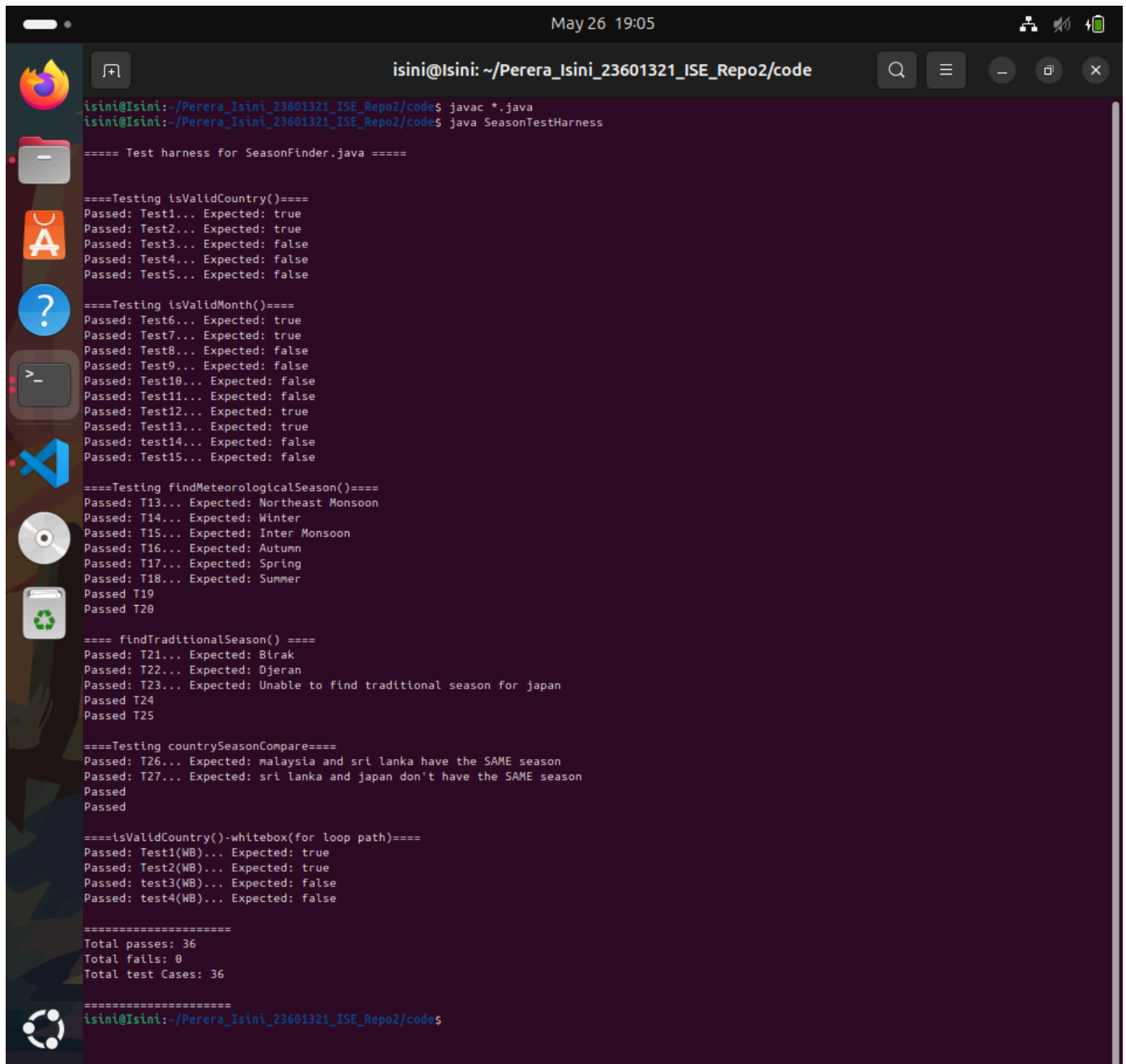
Path	Test Data	Expected Result
1. Enter try path	perthComparison.txt (exists)	output: "Perth Comparison Results"
2. Enter catch path	perthComparison.txt(doesnt exist)	output:"File not found..."

the constructor tested below is *for loops* for 'isValidCountry()' submodule

Path	Test Data	Expected Result
1. Loop finds match on first iteration	australia	true
2. Loop finds match after a few iterations	sri lanka	true
3. Loop completes without finding the match	"Perera"	false
4. Empty country entered, skips loop	""	false

7. Running the test code

7.1 Running the test code for Season



A terminal window titled 'isini@Isini: ~/Perera_Isini_23601321_ISE_Repo2/code' showing the execution of a Java test harness. The user runs 'javac *.java' and 'java SeasonTestHarness'. The output displays test results for 'SeasonFinder.java', including tests for 'isValidCountry()', 'isValidMonth()', 'findMeteorologicalSeason()', 'findTraditionalSeason()', and 'countrySeasonCompare()'. All tests passed, with a total of 36 passes and 0 fails.

```
isini@Isini:~/Perera_Isini_23601321_ISE_Repo2/code$ javac *.java
isini@Isini:~/Perera_Isini_23601321_ISE_Repo2/code$ java SeasonTestHarness

===== Test harness for SeasonFinder.java =====

====Testing isValidCountry()====
Passed: Test1... Expected: true
Passed: Test2... Expected: true
Passed: Test3... Expected: false
Passed: Test4... Expected: false
Passed: Test5... Expected: false

====Testing isValidMonth()====
Passed: Test6... Expected: true
Passed: Test7... Expected: true
Passed: Test8... Expected: false
Passed: Test9... Expected: false
Passed: Test10... Expected: false
Passed: Test11... Expected: false
Passed: Test12... Expected: true
Passed: Test13... Expected: true
Passed: test14... Expected: false
Passed: Test15... Expected: false

====Testing findMeteorologicalSeason()====
Passed: T13... Expected: Northeast Monsoon
Passed: T14... Expected: Winter
Passed: T15... Expected: Inter Monsoon
Passed: T16... Expected: Autumn
Passed: T17... Expected: Spring
Passed: T18... Expected: Summer
Passed: T19
Passed: T20

==== findTraditionalSeason() ====
Passed: T21... Expected: Birak
Passed: T22... Expected: Djeran
Passed: T23... Expected: Unable to find traditional season for japan
Passed: T24
Passed: T25

====Testing countrySeasonCompare====
Passed: T26... Expected: malaysia and sri lanka have the SAME season
Passed: T27... Expected: sri lanka and japan don't have the SAME season
Passed
Passed

====isValidCountry()-whitebox(for loop path)====
Passed: Test1(WB)... Expected: true
Passed: Test2(WB)... Expected: true
Passed: test3(WB)... Expected: false
Passed: test4(WB)... Expected: false

=====
Total passes: 36
Total fails: 0
Total test Cases: 36

=====
isini@Isini:~/Perera_Isini_23601321_ISE_Repo2/code$
```

7.2 Running the test code for Temperature

```

May 27 04:32
isini@isini: ~/Perera_Isini_23601321_ISE_Repo2/code
isini@isini:~/Perera_Isini_23601321_ISE_Repo2/code$ java TemperatureTestHarness
===== Test harness for TemperatureChecker.java =====

===== Testing isValidCty() =====
Passed: Test1... true
Passed: Test2... true
Passed: Test3... false
Passed: Test4... false
Passed: Test5... true

===== Testing isValidTemp() =====
Passed: Test6... true
Passed: Test7... true
Passed: Test8... true
Passed: Test9... false
Passed: Test10... false
Passed: Test11... false

===== Testing compareAvgTemp() =====
Passed: Test12... Temperature is above average by 8.2 degrees C and difference is greater than 6 degrees C
Passed: Test13... Temperature is above average by 2.0 degrees C
Passed: Test14... Temperature is below average by 13.0 degrees C and difference is greater than 6 degrees C
Passed: Test15... Temperature is below average by 5.1 degrees C
Passed: Test16... Temperature and Average is equal
Passed: Test17
Passed: Test18
Passed: Test19

===== Testing comparePerthTemp() =====
Passed: Test20
Passed: Test21
Passed: Test22
Passed: Test23

=====Testing isValidTemp() - BVA=====
Passed: Test24... true
Passed: Test25... true
Passed: Test26... true
Passed: Test27... false
Passed: Test28... false
Passed: Test29... false
Passed: Test30... true
Passed: Test31... true
Passed: Test32... false
Passed: Test33... false
Passed: Test34... false

=====readPerthFile() - MB(try-catch path)=====
===== Perth comparison results =====
Test for the white box test
Passed Test1(MB)
File not found...
Passed Test2(MB)

=====
Total passes: 36
Total fails: 0
Total test cases: 36
=====
isini@isini:~/Perera_Isini_23601321_ISE_Repo2/code$

```

8. Summary table of your work

The following traceability matrix provides a summary of the testing methods, data types, and input/output methods used for each module implemented in the system. It also shows the black-box and white-box testing techniques that were designed, implemented, and executed throughout the assignment.

8.1 Traceability Matrix

Module Name	BlackBox(EP)	BlackBox(BVA)	WhiteBox	Data Types	Form of Input and Output	EP	BVA	White-Box
isValidCountry()	done	not done	done	String, Boolean	parameter, return	done	Not done	done
isValidMonth()	done	done	not done	Integer, Boolean	parameter, return	done	done	not done
findMeteorologicalSeason()	done	not done	not done	String, Integer	parameter, return value	done	not done	not done
findTraditionalSeason()	done	not done	not done	String, Integer	parameter, return	done	not done	not done
countrySeasonCompare()	done	not done	not done	String, Integer	parameter, displayed output	done	not done	not done
isValidCity()	done	not done	not done	String, Boolean	parameter, return	done	is done	is done
isValidTemp()	done	done	not done	String, Double, Boolean	parameter, return	done	done	not done
compareAvgTemp()	done	not done	done	String, Double	parameter, return	done	not done	done
comparePerthTemp()	done	not done	not done	String, Double	parameter, text file output	done	not done	not done

Module Name	BlackBox(EP)	BlackBox(BVA)	WhiteBox	Data Types	Form of Input and Output	EP	BVA	White-Box
readPerthFile()	not done	not done	done	Text file, String	text file input, displayed output	not done	not done	done

9. Version Control

Git version control was used throughout the development process to manage and track changes made to the project files. Separate branches were used during development to organizing coding and documentation. The *master* branch was mainly used as the final branch, while the *report* branch was used to complete the report and related files.

The following screenshots show the branches created and the commit history of the repository.

```

May 27 05:18
isini@Isini: ~/Perera_Isini_23601321_ISE_Repo2
isini@Isini:~/Perera_Isini_23601321_ISE_Repo2$ git branch
* master
+ report
isini@Isini:~/Perera_Isini_23601321_ISE_Repo2$

```

the branches in the repository

```

May 27 05:23
isini@Isini: ~/Perera_Isini_23601321_ISE_Repo2/code
isini@Isini:~/Perera_Isini_23601321_ISE_Repo2/code$ git log
commit ae5d9be36a3085e9090c402c4774e59764b76749 (HEAD -> master)
Author: Isini Perera <isini.perera00@gmail.com>
Date: Tue May 26 13:22:15 2026 +0000
    edited the temperatureChecker.java file based on the errors caught on the testharness file

commit 842faee4de14eaca64074100b4089a1cdf33118
Author: Isini Perera <isini.perera00@gmail.com>
Date: Tue May 26 13:18:31 2026 +0000
    corrected the errors in the temperature test harness

commit e45f82b98ef0b069062c8a81069ad82189d02e4e
Author: Isini Perera <isini.perera00@gmail.com>
Date: Tue May 26 07:41:12 2026 +0000
    created test harness for temperature

commit 259c329e101cad0a057d576a08b258e65c1b0b0b
Author: Isini Perera <isini.perera00@gmail.com>
Date: Tue May 26 07:25:40 2026 +0000
    created perthComparison.txt file for comparison output

commit 00b15a4f2ae564ea09944a1201500ca3074d7a0
Author: Isini Perera <isini.perera00@gmail.com>
Date: Tue May 26 07:23:49 2026 +0000
    created TemperatureChecker.java file

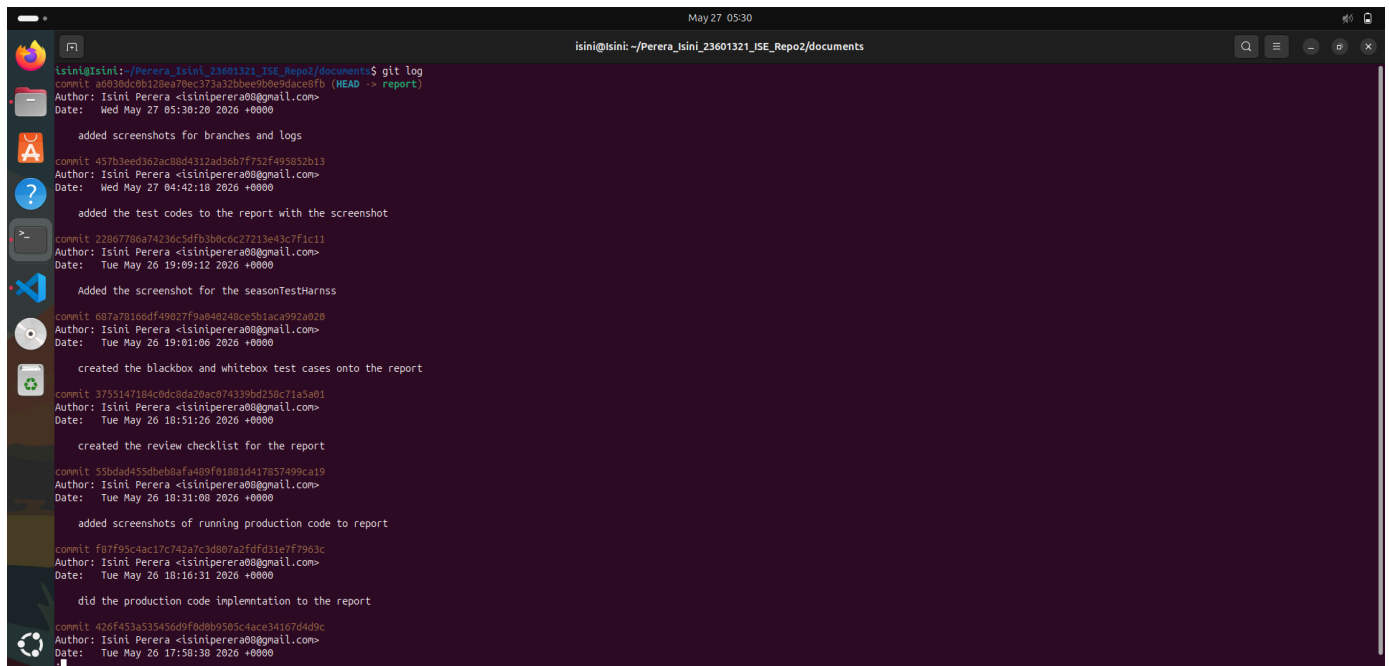
commit 514c46730a4b1b402bf025f4a656546b961bcc1
Author: Isini Perera <isini.perera00@gmail.com>
Date: Tue May 26 07:21:57 2026 +0000
    created SeasonFinder.java file

commit ef05f0dba27cae99a0f185981340094491139d856
Author: Isini Perera <isini.perera00@gmail.com>
Date: Tue May 26 07:18:14 2026 +0000
    renamed Main.java file to Menu.java

commit d939b24d5abdae4b2a9bb29c39fd7aa0dc91fda3
Author: Isini Perera <isini.perera00@gmail.com>
Date: Tue May 26 07:14:33 2026 +0000
    :

```

the git log in master branch



```

isini@isini:~/Perera_Isini_23601321_ISE_Repo2/documents$ git log
commit a6036dcbb128ea78ec373a32bbe9d9de9dace8fb (HEAD -> report)
Author: Isini Perera <isini.perera@gmail.com>
Date: Wed May 27 05:30:20 2026 +0000

    added screenshots for branches and logs

commit 457b3eed362ac8d84312ad36b7f752f495852b13
Author: Isini Perera <isini.perera@gmail.com>
Date: Wed May 27 04:42:18 2026 +0000

    added the test codes to the report with the screenshot

commit 22867786a74236c5dfb3b0c6c27213e43c7f1c11
Author: Isini Perera <isini.perera@gmail.com>
Date: Tue May 26 19:09:12 2026 +0000

    Added the screenshot for the seasonTestHarness

commit 687a78166df49027f9a040248ce5b1aca992a020
Author: Isini Perera <isini.perera@gmail.com>
Date: Tue May 26 19:01:06 2026 +0000

    created the blackbox and whitebox test cases onto the report

commit 3755147184c8dc8da20ac97a339bd458c71a5501
Author: Isini Perera <isini.perera@gmail.com>
Date: Tue May 26 18:51:26 2026 +0000

    created the review checklist for the report

commit 55bdad455d5eb0dafa489f01801d417857499ca19
Author: Isini Perera <isini.perera@gmail.com>
Date: Tue May 26 18:31:08 2026 +0000

    added screenshots of running production code to report

commit f87f954ac17c742a7c3d807a2fdd31e7f7963c
Author: Isini Perera <isini.perera@gmail.com>
Date: Tue May 26 18:16:31 2026 +0000

    did the production code implementation to the report

commit 426f453a535450d9f8db9585c4ace34167d4d9c
Author: Isini Perera <isini.perera@gmail.com>
Date: Tue May 26 17:58:38 2026 +0000

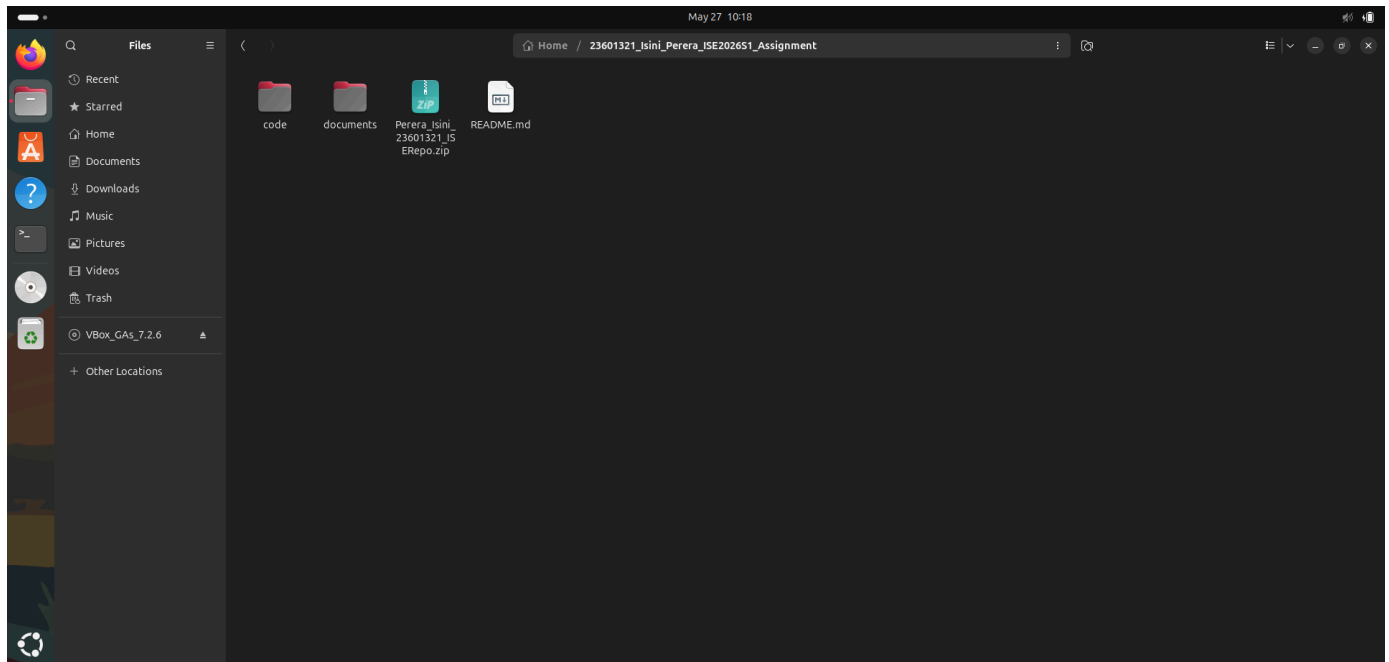
```

the git log in report branch

9.1 Repository Issue Faced During Development

During the final stages of the assignment, the original Git repository became corrupted due to Git object and HEAD reference errors. This caused the commit history and version control information in the original repository to become inaccessible. After reporting the issue and following the advice given, all project files were backed up safely and a new repository was recreated in order to continue the remaining development and documentation work.

For reference purposes, the corrupted repository files have also been included separately in the submission folder as instructed.



The following screenshot shows the corrupted repository backup file included in the submission folder for reference purposes.

10. Discussion

Through this assignment, I was able to learn how to design and implement a modular software system using Java. Different modules and submodules were created for the SeasonFinder and TemperatureChecker classes by applying modularity concepts learnt during lectures and worksheets. I also gained experience in designing and executing black-box and white-box test cases using Java test harnesses.

One of the main challenges faced during the assignment was that the original Git repository became corrupted close to the submission date, which caused the commit history to become inaccessible. To continue the work safely, all project files were backed up and a new

repository was recreated. This experience helped me understand the importance of maintaining backups and using version control carefully.

Overall, the assignment helped improve my understanding of modular design, testing methods, debugging, and version control. In the future, the system can be improved further by adding more countries, cities, and a graphical user interface to make the program more user-friendly.